

Технічний Roadmap Backend-частини

Стратегія розвитку платформи після стадії MVP

1. Вступ та поточний стан

На стадії MVP (Minimum Viable Product) backend-архітектура Sellora побудована як монолітний Spring Boot додаток з використанням PostgreSQL як єдиного джерела даних (Source of Truth). Усі клієнтські запити обробляються синхронно через REST API.

Ця архітектура дозволила швидко розробити продукт та перевірити бізнес-гіпотези. Однак, із зростанням кількості користувачів та специфіки нашого домену (Групові покупки з таймерами та спільним доступом), поточна модель має обмеження.

Цей документ описує конкретні технічні кроки (без переходу на повноцінні мікросервіси) для розв'язання проблем продуктивності та покращення User Experience.

2. Впровадження WebSockets / STOMP для реального часу

2.1. Проблема: HTTP-полінг

Основна фіча Sellora — "Групові покупки", які обмежені в часі та вимагають збору певної кількості учасників (наприклад, 3 людини за 24 години). Наразі фронтенд змушений періодично надсилати HTTP-запити (полінг) до сервера, щоб перевірити, чи приєднався хтось новий, або чи змінився статус групи.

Це створює дві проблеми:

1. **Затримки в UI:** Користувач може не бачити актуальної інформації кілька секунд.

2. **Навантаження на сервер:** Тисячі клієнтів, які кожні 5 секунд смикають БД для перевірки статусу сесії, створюють значне навантаження на пул з'єднань PostgreSQL.

2.2. Рішення: Spring WebSockets + STOMP

Ми плануємо впровадити протокол WebSocket у зв'язці з підпротоколом STOMP (Simple Text Oriented Messaging Protocol) як Message Broker.

Як це працюватиме:

- При вході на сторінку товару або деталі групової сесії, клієнт (Vue.js) встановлює стійке з'єднання через WebSocket та підписується на конкретний канал, наприклад: `/topic/group-sessions/{uuid}`.
- Коли інший користувач приєднується до цієї сесії, наш існуючий `PaymentService` або `GroupBuySessionService` надсилає повідомлення у цей канал через `SimpMessagingTemplate`.
- Всі підписані клієнти миттєво отримують оновлення: зміну кількості учасників, закриття збору або оновлення таймера, *без необхідності робити окремий запит до БД.*

Цей крок кардинально зменшить кількість марних HTTP-запитів і забезпечить справжній "живий" (viral) соціальний досвід для користувачів.

3. Інтеграція Redis Caching

3.1. Проблема: Читання з БД

Маркетплейси типово мають співвідношення операцій читання до запису як 100:1. Наша PostgreSQL база даних обробляє кожен запит на отримання каталогу товарів, фільтрацію категорій та перевірку деталей товарів.

Крім того, сесії групових покупок постійно запитуються фронтом. У період пікових навантажень (маркетингові кампанії, розсилки) це може призвести до деградації продуктивності бази даних.

3.2. Рішення: Розподілений кеш на базі Redis

Впровадження Redis (In-Memory Data Store) як проміжного шару кешування (`@EnableCaching` у Spring Boot).

Ключові зони кешування:

1. Каталог товарів та Категорії (Read-Heavy):

- Відповіді від `/categories/tree` та часто запитувані товари (`/products/{id}`) будуть кешуватися в Redis.
- Кеш інвалідується (видаляється) автоматично лише тоді, коли продавець оновлює дані товару (через `@CacheEvict` у методі `updateProduct`).

2. Облік переглядів/Статистика:

- Замість того, щоб інкрементувати лічильник переглядів товару напряму в PostgreSQL при кожному заході, ми збиратимемо ці інкременти в Redis.
- Окремий Quartz Scheduler раз на хвилину скидатиме агреговані дані (Batch Update) у PostgreSQL, зберігаючи ресурси реляційної БД.

3. Rate Limiting та Безпека:

- Навіть без впровадження комерційних тарифів, Redis дозволить легко налаштувати технічний Rate-limiting (захист від DDoS/скрапінгу), використовуючи алгоритм Token Bucket для певних IP або ідентифікаторів.

4. Підсумки

Реалізація цих двох кроків (WebSockets + Redis) є стратегічно важливішою на етапі після MVP, ніж передчасне розбиття моноліту на мікросервіси.

Ці зміни забезпечать:

- **Чуйний інтерфейс (Snappy UI)** завдяки миттєвим сповіщенням через STOMP.
- **Горизонтальну масштабованість (Scalability)** завдяки розвантаженню PostgreSQL.
- **Зменшення інфраструктурних витрат (Cost Efficiency)**, оскільки один сервер зможе тримати в десятки разів більше одночасних користувачів.